Talk-to-Data Delivery Blueprint — Phase 4: Design Architecture

Daniel Brule · linkedin.com/in/danielbrule
v1.0 · June 2026

1 Executive summary

Phase 4 designs the GenAI orchestration layer that turns a user question into a governed Talk-to-Data answer. It does not design the user interface, rebuild the data foundation or define the full enterprise architecture. Its scope is the runtime flow from question interpretation through metadata grounding, model use, tool calls, query generation, validation, execution hand-off, answer generation, follow-up handling, evaluation and quality monitoring.

The phase takes the governed foundation from Phase 3 as its main input. It should define how foundation artefacts are retrieved, interpreted and applied at runtime, including the minimum metadata contract required for safe orchestration. Where the foundation is incomplete, ambiguous or not structured for runtime use, the response should be to refine the foundation, narrow scope or defer unsupported questions, not to compensate with prompts.

The central discipline is to reduce uncontrolled model judgement. The model may help interpret intent, retrieve relevant

context, draft queries and explain results, but it should not invent metric logic, choose unsafe joins, bypass permissions, execute unrestricted queries or present uncertain answers as fact.

Phase 4 should also define how the orchestration will be evaluated before build. Model and flow choices should be assessed across correctness, retrieval quality, query validity, safe failure, consistency, latency, cost, maintainability and data exposure. The aim is not to select the most powerful model by default, but to choose the safest and most proportionate model pattern for each task.

The phase must also design how quality will be tracked over time. A T2D system will degrade if changes in data, metadata, prompts, models, user behaviour or business definitions are not monitored. Feedback capture, usage analytics, failed validations, refusals, escalations, answer corrections and regression tests should be designed into the orchestration from the start.

By the end of Phase 4, stakeholders should be able to approve a clear orchestration design: how questions become answers, how follow-up conversations are managed, where the model is constrained, how tools are bounded, how quality is evaluated, and what evidence is needed before building the bounded prototype.

The main evidence before progressing is an approved orchestration design, model and tool boundaries, validation approach, conversation-handling rules, evaluation design, quality-monitoring approach, security requirements, logging requirements, orchestration governance and integration assumptions.

**
**

2 Phase overview

Phase 4 designs the GenAI orchestration layer for the scoped Talk-to-Data capability. It defines how the system moves from a user question to a governed answer, using the foundation artefacts created in Phase 3.

The focus is the runtime flow between the user question, model, metadata, tools, queryable data layer and final answer. For a POC, the design may be lightweight, but the main orchestration choices should still be explicit. For an MVP, pilot or production path, model use, tool boundaries, validation, logging, conversation state, evaluation and quality monitoring must be clear enough to guide build and assurance.

2.1 Objective

The objective of Phase 4 is to define the orchestration design required to turn governed data foundations into safe conversational answers. It should confirm:

- **Question-to-answer flow:** how intent, context, metadata, query generation, validation, execution and answer generation fit together.
- **Metadata grounding:** how Phase 3 artefacts are retrieved, interpreted and applied.
- **Model and tool strategy:** which models and tools are used, where their boundaries are, and how outputs are constrained.
- **Query and response controls:** how queries are checked and how answers are caveated, refused or escalated when needed.
- **Conversation handling:** how follow-ups reuse context, clarify or re-query.
- **Evaluation and monitoring:** how quality, cost, latency, feedback, usage and drift will be assessed over time.
- **Orchestration governance:** how model, prompt, tool, validation and quality changes are owned and approved.
- **Security constraints:** how access, exposure, retention and audit shape orchestration.

The output is an approved orchestration design and governance model that can guide bounded prototype build, evaluation, security review and later production planning.

2.2 Scope of the phase

Phase 4 remains bounded to the agreed T2D orchestration scope. It does not design the full UI, rebuild the data foundation or define the full enterprise architecture.

In scope are orchestration flow, metadata retrieval, model and prompt strategy, tool boundaries, query generation,

validation, execution hand-off, answer generation, safe failure, conversation handling, logging, evaluation design and quality monitoring.

The phase should confirm the minimum metadata contract required from Phase 3. If the required metric, dimension, join, caveat, access rule or example is not available in a usable form, the design should route it back for remediation, narrowing or deferral.

The phase also sets the model-pattern, cost and evaluation assumptions that will guide prototype build and later model comparison.

2.3 What this phase does not do

Phase 4 does not build the prototype. Its role is to define the orchestration design that Phase 5 will implement in a bounded way. It also does not create the governed data foundation; that is the purpose of Phase 3. However, it may identify gaps in foundation artefacts that need to be resolved before they can support runtime orchestration.

Phase 4 does not design the full user interface either. It may define interaction assumptions, such as support for follow-up questions, clarification, refusals, feedback capture or source visibility, but detailed UX design sits outside this phase.

2.4 Expected duration and level of effort

The effort required depends on delivery maturity, risk and technical complexity.

For a narrow POC, Phase 4 may be completed in a short design sprint if the Phase 3 foundation is clear and the target flow is simple. For an MVP, pilot or production path, the team should define model/task allocation, metadata retrieval patterns, validation layers, tool permissions, logging, conversation state, quality monitoring, cost and latency assumptions.

The phase is complete when the team has enough design clarity to build a bounded prototype without making critical orchestration, safety or evaluation decisions ad hoc during implementation.

Some Phase 5 work may start early, such as scaffolding or test harnesses, but it should not lock in core orchestration decisions before Phase 4 approval.

2.5 Main participants and decision owners

Phase 4 requires AI, data, governance and business input but it should not be owned by the model engineer alone, because the key decisions concern governed metadata, access, validation, observability and business trust.

Role	Main responsibility in Phase 4
Product owner	Confirms scope boundaries, interaction assumptions and prototype priorities.
AI / solution architect	Owns orchestration design, model/tool flow and design trade-offs.
Data / semantic owner	Confirms how Phase 3 artefacts are consumed and where metadata gaps remain.
Analytics / data engineer	Advises on query execution, performance, cost and technical constraints.
Security / governance lead	Reviews access, exposure, retention, audit, tool permissions and safe failure.
Evaluation owner	Defines test sets, quality measures, model comparison and regression needs.
Business SME	Validates ambiguity handling, caveats, explanations and failure modes.
Operating owner	Reviews monitoring, supportability, feedback handling and ownership implications.

A T2D system should not proceed to prototype build without clear accountability for model behaviour, metadata grounding, tool boundaries, validation, answer quality and monitoring.

3 Readiness decision and delivery implications

Phase 4 should not end with an informal view that the architecture “looks sensible”. It should end with a clear decision on whether the orchestration design is strong enough to support bounded prototype build.

The decision should be based on the approved flow, model and tool boundaries, metadata contract, validation approach, conversation-handling rules, logging requirements and evaluation design. The aim is to avoid entering prototype build while critical runtime decisions are still being improvised.

3.1 Possible Phase 4 outcomes

Phase 4 should end with one of five outcomes:

Outcome	Meaning
Proceed	The orchestration design is clear enough to support bounded prototype build.
Proceed with constraints	Build can start, but only within agreed limits, such as restricted questions, users, tools or data scope.
Refine foundation	Phase 3 artefacts are not structured enough for retrieval, validation or answer explanation.
Redesign	The proposed orchestration flow, model route, tool boundary or validation pattern is not safe or practical enough.
Pause	Material risks remain unresolved, such as access exposure, weak validation, unclear ownership, excessive cost or no evaluation route.

A proceed decision is not approval for production. It means the design is credible enough to build and test a bounded end-to-end prototype.

3.2 Minimum conditions to proceed

Phase 4 should not progress to prototype build unless the team can confirm:

- The end-to-end question-to-answer flow is defined.
- Phase 3 artefacts can be retrieved or packaged for runtime use.
- Model roles and tool boundaries are explicit.
- Query generation and validation controls are defined.
- Answer caveats, refusals and escalation rules are agreed.
- Conversation state and follow-up handling are understood.
- Logs and traces are sufficient to diagnose failures.
- Model and orchestration evaluation can be run.
- Orchestration governance and change ownership are defined.
- Security constraints for model use, tools, logs and exposure are defined.
- Unresolved risks, assumptions and constraints have owners.

The standard does not need to be production-grade for a POC, but the design should be explicit enough that the prototype tests the intended pattern rather than inventing it during build.

3.3 Common reasons to narrow, refine, redesign or pause

Common reasons to avoid moving forward unchanged include:

- **Prompt-dependent governance:** metric logic, joins, filters or caveats still rely on prompt interpretation.
- **Weak metadata contract:** Phase 3 artefacts are documented but not usable for reliable runtime grounding.
- **Over-broad tool access:** tools can reach too much data, execute too freely or are poorly logged.
- **Shallow validation:** checks focus on syntax but miss permissions, scope, joins, cost or plausibility.
- **Unsafe follow-ups:** conversation context can silently change filters, periods, entities or scope.
- **Overconfident answers:** partial, uncertain or caveated results can be presented as fact.
- **Unviable performance:** expected latency or cost does not support the intended user journey.
- **No change control:** prompts, models, tools, validation rules or thresholds can change without clear approval.
- **Unclear security constraints:** model inputs, tool permissions, logs or traces expose data unnecessarily.

The right response is not always to stop. In many cases, the better decision is to narrow the question set, restrict tools, simplify conversation handling, strengthen metadata, add validation or defer unsupported interactions.

3.4 How Phase 4 shapes later phases

Phase 4 sets the build constraints for Phase 5. It should tell the prototype team what flow to implement, which tools are allowed, what metadata is required, which validation checks are mandatory, how conversations should behave, what

evidence must be captured, and how orchestration changes will be governed after the prototype.

It also shapes later evaluation, pilot and production readiness. Weak Phase 4 design usually reappears later as unreliable answers, unclear refusals, hard-to-debug failures, expensive model usage, unsafe tool access or no credible way to explain why an answer was produced.

A strong Phase 4 does not remove all uncertainty. It makes the uncertainty testable.

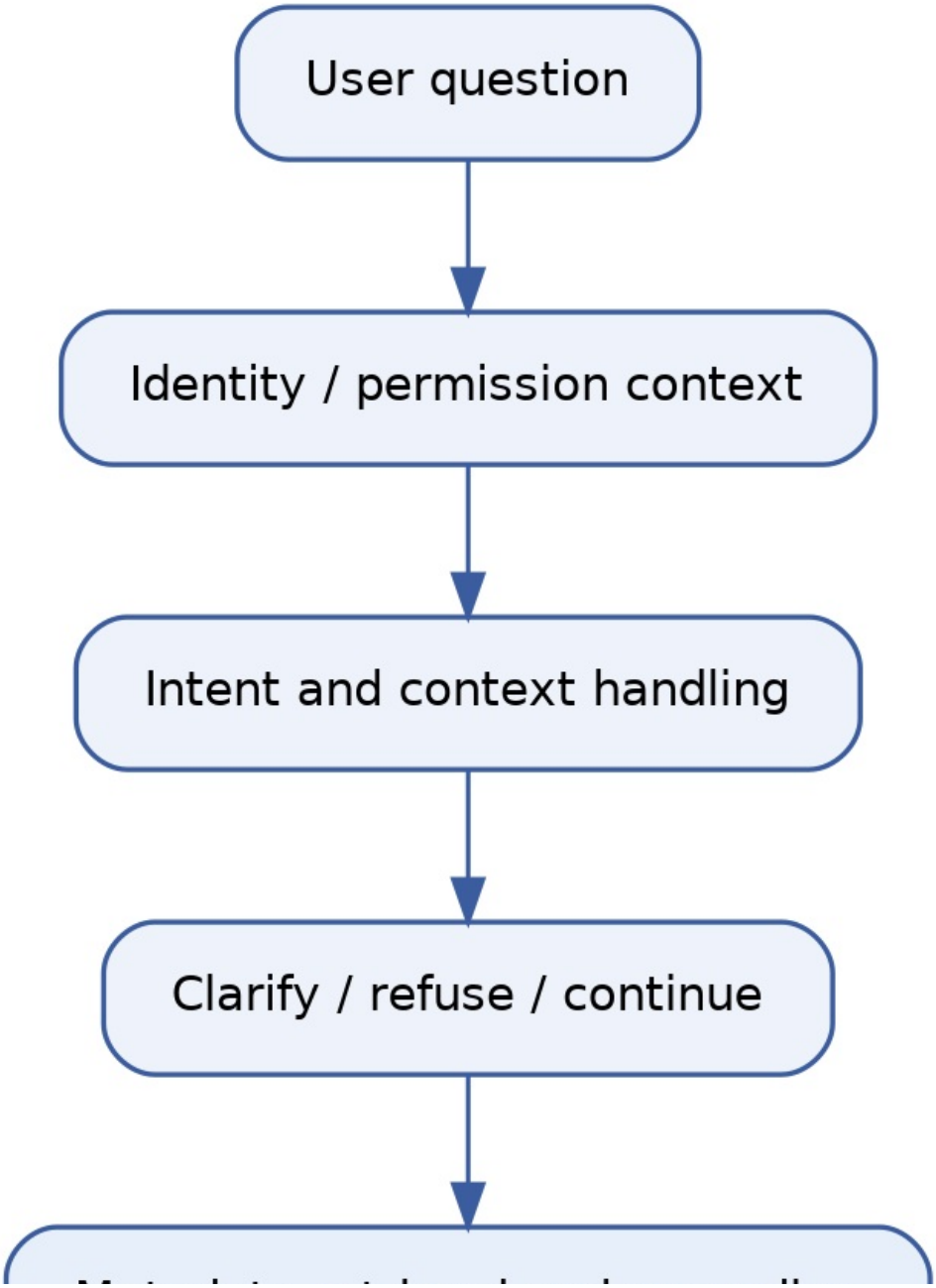
Phase 4 should also make clear which operational scaffolding Phase 5 must implement early, including logging, tracing, prompt and model versioning, cost and latency capture, basic deployment discipline and governance controls.

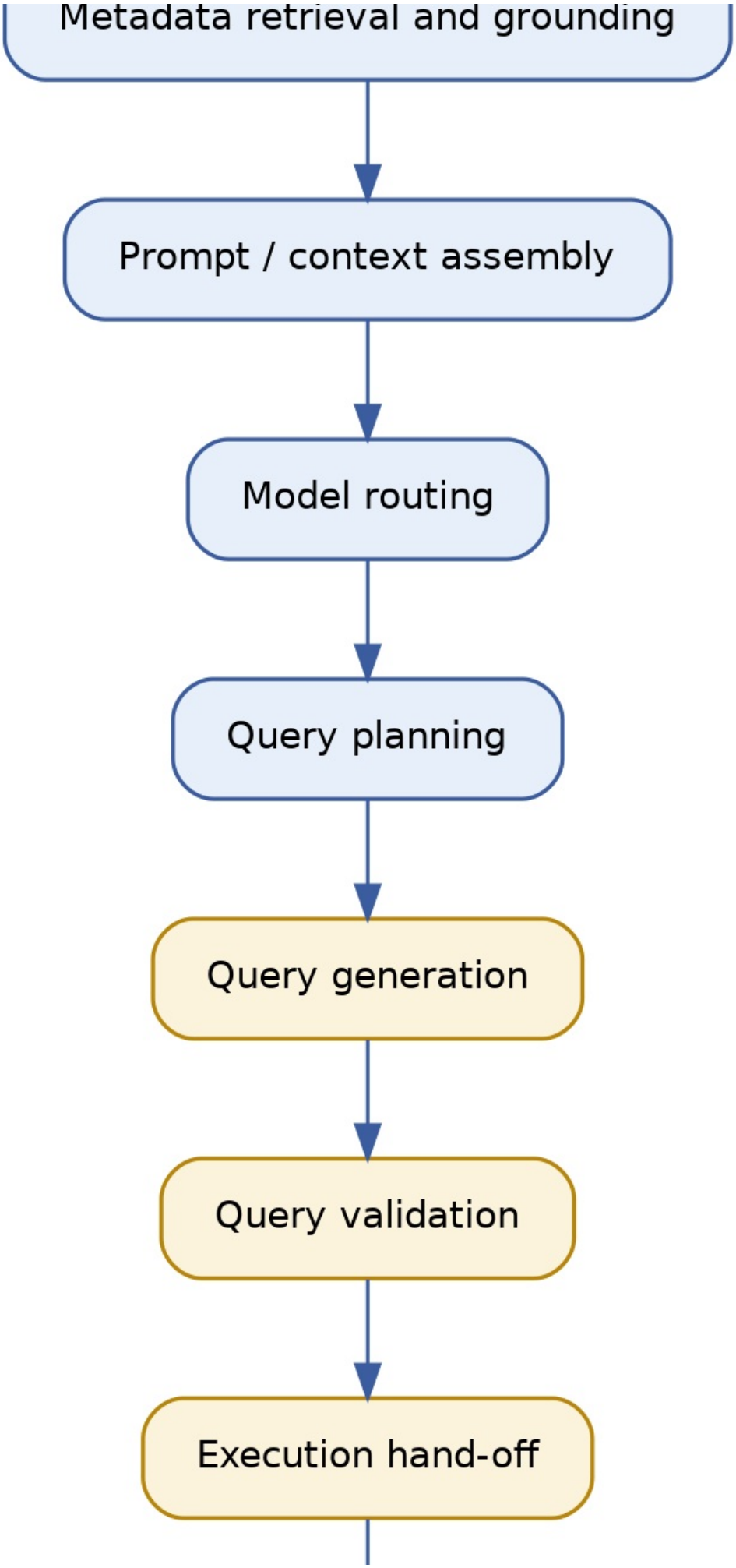
4 Orchestration design activities overview

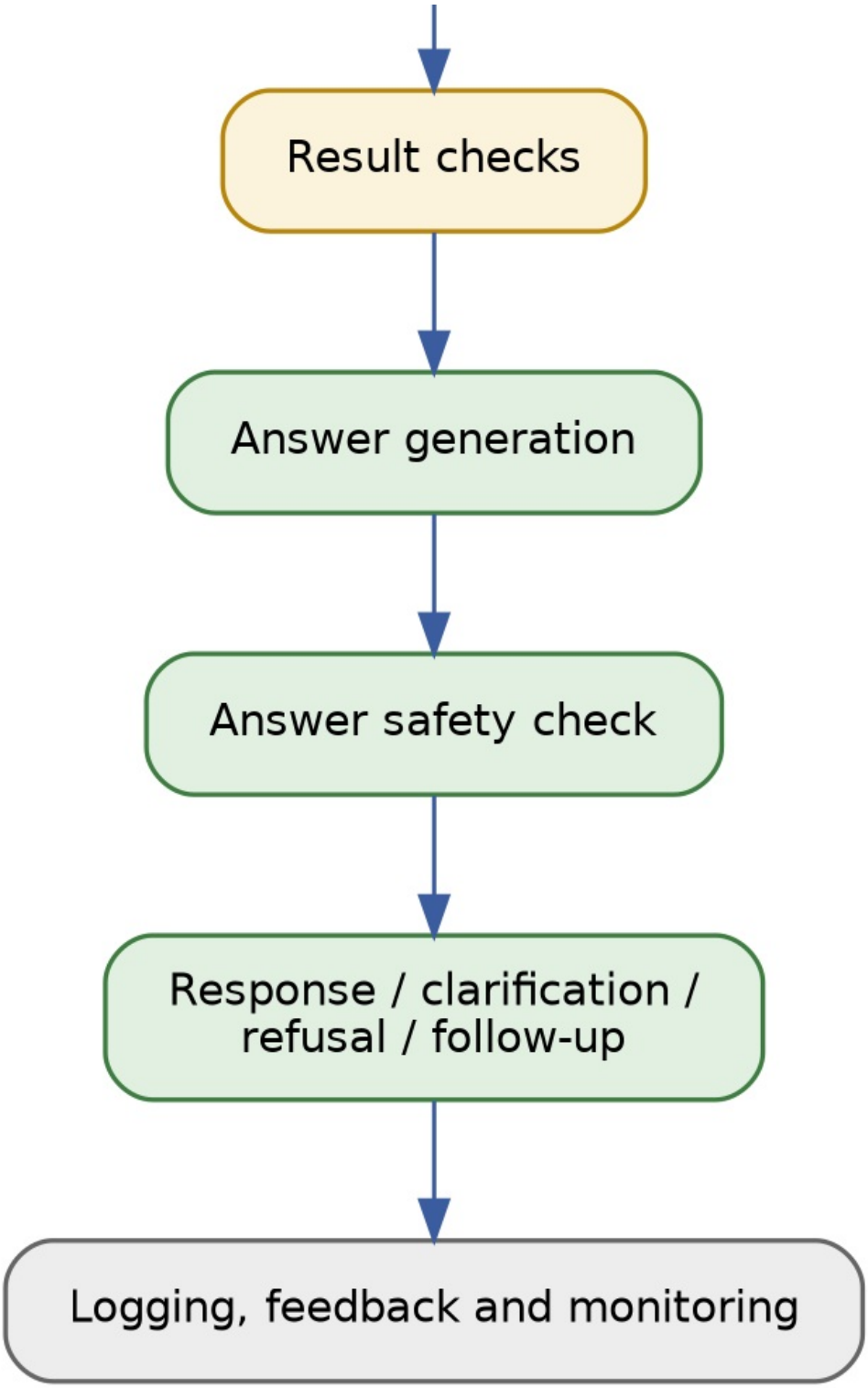
Phase 4 should translate the orchestration decision into a clear design path. The activities are ordered from principles and flow design through metadata grounding, model and tool choices, validation, answer behaviour, conversation handling, evaluation, monitoring and governance.

4.1 End-to-end orchestration flow

The flow below shows how Phase 4 connects a user question to a governed answer. It is not a full technical architecture diagram; it is the orchestration logic that the detailed design should validate. It can be simplified depending on project scope and delivery stage.




Metadata retrieval and grounding
Prompt / context assembly
Model routing
Query planning
Query generation
Query validation
Execution hand-off



Orchestration flow

The flow above is the main path. Clarification, refusal, validation failure, low-confidence handling and escalation are not separate diagrams; they are decision points *within* these stages and are detailed in the activities below.

4.2 Activity sequence

Activity	Main question	Main output
1. Confirm orchestration principles and constraints	What design rules must the orchestration follow?	Approved principles, constraints and security assumptions.
2. Define question-to-answer flow	How does a question become a governed answer?	End-to-end orchestration flow.
3. Design metadata retrieval and grounding	How will governed context be retrieved and applied?	Metadata retrieval and grounding design.
4. Define model, prompt and routing strategy	Which models support which orchestration	Model/task strategy and prompt approach.

	tasks?	
5. Define tool boundaries and execution hand-off	What tools can be called, and under what limits?	Tool registry and execution rules.
6. Design query generation and validation	How are queries created, checked and controlled?	Query generation and validation pattern.
7. Design answer generation and safe failure	How are results explained, caveated or refused?	Answer and safe-failure rules.
8. Design multi-turn conversation handling	How are follow-up questions managed?	Conversation state and follow-up rules.
9. Design model and orchestration evaluation	How will quality and safety be tested?	Evaluation dimensions and test approach.
10. Design quality monitoring and improvement loop	How will quality be tracked after build?	Feedback, monitoring and improvement design.
11. Define orchestration governance and change control	Who owns changes to models, prompts and tools?	Governance and change-control model.

4.3 Activity logic

The activities should not be treated as independent workstreams. Decisions made in one area affect the others.

Metadata retrieval shapes prompt design and query generation. Tool boundaries affect validation and logging. Conversation handling changes how context is retained, re-used or rejected. Evaluation design should influence model choice, safe-failure rules and quality monitoring. Governance should make clear how these elements can be changed after the prototype without weakening control.

The key discipline is to avoid designing around a single happy path. Phase 4 should explicitly cover ambiguity, unsupported questions, unsafe joins, missing metadata, permission failures, high-cost queries, slow responses, uncertain results and user corrections.

4.4 Delivery-depth expectations

Output	POC	MVP / pilot	Production path
Orchestration flow	Lightweight but explicit	Approved and testable	Versioned and governed
Metadata contract	Simple structured artefacts	Stable IDs and usage rules	Catalogue, API or governed service
Model and prompt strategy	Task-level choices	Tested model/task routing	Change-controlled and monitored
Tool boundaries	Restricted tool list	Logged tool permissions	Policy-controlled tool access
Validation pattern	Core checks	Layered validation	Automated and auditable controls
Conversation handling	Limited session logic	Defined follow-up rules	Governed state and retention rules
Evaluation design	Small test set	Broader regression set	Continuous evaluation and monitoring
Quality monitoring	Basic feedback capture	Usage and failure tracking	Operational quality dashboard
Governance model	Named owners	Change approvals	Formal operating control

4.5 Practitioner note

The first orchestration flow can be simple, but it should not be brittle. Phase 4 should design modular steps for metadata retrieval, model routing, validation, execution, answer generation, logging and feedback, so later changes can be made without redesigning the whole system.

The first design will not be perfect. Evaluation, monitoring and governance should therefore be linked from the start, so the team can see when the system is drifting, which part of the flow needs improvement, and who is authorised to approve the change.

5 Core orchestration design activities

The activities in this chapter define the design decisions required before building a bounded T2D prototype. They should be applied proportionately: a POC may need lightweight decisions and simple artefacts, while an MVP, pilot or production path requires stronger evidence, clearer ownership and more formal controls.

The aim is not to design a perfect architecture on paper. It is to define a controlled orchestration pattern that can be built, tested, monitored and improved. Each activity should produce enough evidence to guide Phase 5 build without leaving critical model, metadata, tool, validation, security or governance decisions to be improvised during implementation.

5.1 Confirm orchestration principles and constraints

Purpose: Define the non-negotiable design rules for the orchestration layer before model, tool, validation and monitoring choices are made.

Key design decisions

- Which business, data, security and technology constraints must the orchestration respect?
- What should the model decide, and what must be governed outside the model?
- Which data, metadata, query text, results and logs may be shared with the model?
- What explainability, auditability and traceability are required?
- What user, cost, scale and performance constraints should shape the design?
- Which deployment, region, retention and data-transfer constraints apply?
- What changes require approval after prototype build?

Typical outputs

- Orchestration principles.
- Security, exposure and deployment assumptions.
- Model and tool-use constraints.
- Logging and traceability assumptions.
- Cost, latency and scale assumptions.
- User-need and trust assumptions.
- Change-control assumptions.
- Open design risks and decisions.

Red flags

- Principles are generic and do not affect design choices.
- The model is expected to infer governed business logic.
- Security, exposure or deployment rules are unclear.
- Tool access is assumed before permissions are agreed.
- Cost and scale constraints are ignored.
- The flow is technically elegant but weakly linked to user need.
- Logs are treated as optional.
- No one owns future orchestration changes.

Practitioner note

Note 1: The goal is to answer a business need, not to showcase a model. User need, decision value and trust requirements should come before model choice, tool complexity or architectural elegance.

Note 2: Auditability and security become especially important in regulated environments or where sensitive personal data is involved. However, even in lower-risk contexts, the main orchestration risks often sit in answer generation and follow-up handling: the system may overstate results, lose caveats, inherit the wrong context or move outside the approved scope.

Note 3: Cost is an architectural constraint, not only a commercial concern. A design that relies on the strongest model, broad retrieval and unrestricted queries for every question is not a scalable T2D pattern.

5.2 Define question-to-answer flow

Purpose: Define the end-to-end flow that turns a user question into a governed answer, including clarification, validation, execution and response behaviour.

Key design decisions

- What are the main steps from user question to final answer?
- When should the system retrieve metadata, call tools or ask for clarification?
- When should a query be generated, validated, executed or blocked?
- How should the system handle ambiguous, unsupported or out-of-scope questions?
- What evidence should be captured at each step?
- Which parts of the flow can be simplified for the first prototype?

Typical outputs

- End-to-end orchestration flow.
- Decision points and failure paths.
- Clarification and refusal rules.
- Execution and validation sequence.
- Logging points across the flow.
- Prototype flow boundary.

Red flags

- The flow only covers the happy path.
- Clarification and refusal points are unclear.
- Validation happens too late or not at all.
- Execution can occur before scope or permission checks.
- Follow-up questions can bypass earlier controls.
- The prototype flow cannot evolve into a safer pattern.

Practitioner note

The flow should start from the business question, not from the model call. A good T2D flow makes clear how the system protects the user from confident wrong answers, not only how it generates a response.

5.3 Design metadata retrieval and grounding

Purpose: Define how the orchestration layer retrieves and applies governed metadata before query generation, validation and answer generation.

Key design decisions

- Which Phase 3 artefacts are required at runtime?
- Which metadata is retrieved dynamically, packaged at build time or embedded in prompts?
- How are metrics, dimensions, joins, filters, caveats and examples selected?
- How are metadata versions, owners and approval status checked?
- What happens when metadata is missing, stale, conflicting or incomplete?
- How is retrieved metadata linked to logs, answers and evaluation evidence?

Typical outputs

- Metadata retrieval design.
- Phase 3 metadata contract.
- Grounding rules for query generation.

- Grounding rules for answer generation.
- Metadata versioning assumptions.
- Missing-metadata handling rules.
- Metadata risks and remediation actions.

Red flags

- The model relies on loose documentation or prompt memory.
- Metric logic, joins or caveats are not retrievable in a structured way.
- Metadata has no stable IDs, owners or versions.
- Conflicting definitions can be retrieved without resolution rules.
- Missing metadata leads to guessing instead of clarification or refusal.
- Retrieved metadata is not visible in logs or evaluation evidence.

Practitioner note

Metadata retrieval is often a good place to use smaller, embedded or cheaper models, because the task can be constrained and evaluated. However, retrieval errors can corrupt the whole flow. The design should favour the simplest model pattern that reliably retrieves the right governed context, not the cheapest one that appears to work in a demo.

5.4 Define model, prompt and routing strategy

Purpose: Define which models are used for each orchestration task, how prompts are controlled, and when the flow should route between models, tools or deterministic rules.

Key design decisions

- Which tasks need a model, and which should use rules or tools?
- Which model pattern is appropriate for routing, retrieval, SQL, validation and answers?
- Which deployment route is allowed: hosted, private cloud, VPC, on-premise or approved SaaS?
- Which region, residency, retention and cross-border transfer constraints apply?
- Can prompts, metadata, query text, results or logs be used for model training or provider improvement?
- What context is included in prompts, and what is excluded?
- When should the system use a stronger model, cheaper model or fallback route?
- How are prompt versions, model versions and routing decisions logged?

Typical outputs

- Model/task strategy.
- Prompt approach and constraints.
- Routing and fallback rules.
- Model deployment assumptions.
- Region, retention and training-use assumptions.
- Model pattern and evaluation shortlist.
- Prompt and model logging requirements.

Red flags

- The strongest model is used for every task by default.
- Prompts contain more data than the task requires.

- Model region, retention or training-use settings are unclear.
- Routing decisions are not logged or explainable.
- Prompt changes can be made without review.
- Cross-border transfer constraints are not understood.
- Fallback behaviour is undefined.
- Model choice is not tested against cost, latency and quality.

Practitioner note

Model strategy should be task-led, not provider-led. In a T2D system, smaller or embedded models may be enough for routing and metadata ranking, while stronger models may be justified for complex query generation or answer synthesis. The right design is usually a controlled mix, not one model doing everything.

5.5 Define tool boundaries and execution hand-off

Purpose: Define which tools the orchestration layer can call, what each tool is allowed to do, and how execution is handed off safely.

Key design decisions

- Which tools are needed for metadata, validation, execution and logging?
- What can each tool read, write, execute or return?
- Which tools run under user identity, service identity or delegated access?
- What permissions, row limits, timeouts and cost limits apply?
- What should happen when a tool fails, times out or returns unsafe output?
- Which tool calls, inputs, outputs and decisions must be logged?

Typical outputs

- Tool registry.
- Tool permission model.
- Execution hand-off pattern.
- Tool input and output rules.
- Timeout, row-limit and cost rules.
- Tool failure and escalation rules.
- Tool logging requirements.

Red flags

- Tools have broader access than the T2D use case requires.
- SQL execution is available before validation.
- Tool calls are not linked to user identity or permissions.
- Tool failures return raw errors directly to users.
- Tool inputs or outputs expose unnecessary sensitive data.
- There is no timeout, row-limit or cost-control pattern.
- Tool activity cannot be reconstructed from logs.

Practitioner note

Typical T2D tools include metadata retrieval, query planning, SQL/query validation, query execution, permission checking, result checking, logging/tracing and user feedback capture. Keep each tool narrow, explicit and logged; avoid one broad

“data tool” that can retrieve, generate, validate and execute without clear boundaries.

5.6 Design query generation and validation

Purpose: Define how queries or tool calls are generated, checked and controlled before execution.

Key design decisions

- What inputs are allowed for query generation?
- How are approved metrics, joins, filters and limits applied?
- Which query types are allowed, restricted or blocked?
- Which validation checks must pass before execution?
- When should the system revise, clarify, refuse or escalate?
- How are query versions, validation results and failures logged?
- What cost, runtime, scan and timeout limits apply before execution?

Typical outputs

- Query generation pattern.
- Approved query constraints.
- Validation rule set.
- Query revision rules.
- Blocking and escalation rules.
- Query logging requirements.
- Known query risks and limits.
- Query cost and timeout rules.

Red flags

- The model can generate SQL against unrestricted schema.
- Validation only checks syntax.
- Unsafe joins or grains can reach execution.
- Mandatory filters or row limits are missing.
- Failed validation triggers endless regeneration.
- Query cost, timeout or scan limits are undefined.
- Query evidence is not logged.
- Long-running or high-cost queries are not blocked before execution.

Practitioner note

Note 1: Query validation can combine deterministic rules and AI-assisted review, but obvious controls should be rule-based. Write, edit, delete, drop, unrestricted joins, missing row limits, forbidden tables and permission failures should be blocked by deterministic checks before any query reaches execution.

Note 2: Query runtime cannot always be predicted exactly, but the system should still use dry runs, explain plans, scan estimates, timeouts and cost thresholds where available. A T2D system should not allow open-ended analytical queries to run unchecked.

5.7 Design answer generation and safe failure

Purpose: Define how query results are turned into user-facing answers, including caveats, assumptions, refusals, clarifications and escalation.

Key design decisions

- What should the answer include: number, table, chart, explanation, source, caveat or SQL?
- How are metric definitions, time periods, filters and assumptions shown?
- When should the system caveat, refuse, clarify or escalate?
- How should the answer avoid overstating uncertain or partial results?
- What result checks are needed before answer generation?
- How are answer versions, caveats and refusal reasons logged?

Typical outputs

- Answer-generation rules.
- Result interpretation rules.
- Caveat and assumption rules.
- Refusal and escalation rules.
- Answer format assumptions.
- Answer safety checks.
- Answer logging requirements.

Red flags

- The model can explain results without approved caveats.
- Answers hide filters, periods or metric definitions.
- Empty, stale or partial results are treated as complete.
- Refusals are inconsistent or too vague.
- Follow-up answers lose prior assumptions.
- The system sounds more certain than the evidence supports.
- Answer issues cannot be traced back to source, query or metadata.

Practitioner note

Note 1: Users should be able to see the metadata used to support the answer, such as the metric definition, period, filters, caveats and source. Showing the generated SQL can also improve trust for technical users, but it should be optional and governed. Trust increases when users can understand how the answer was produced, not just read the final number.

Note 2: If a confidence signal is shown, it should use a simple scale and wording that non-expert users can understand. Low-confidence answers should trigger clarification, refusal or escalation, not a weak answer with a disclaimer.

5.8 Design multi-turn conversation handling

Purpose: Define how follow-up questions reuse context, trigger clarification, require new queries or safely end the conversation.

Key design decisions

- Which context can be inherited across turns?
- When should inherited assumptions be restated?
- When does a follow-up need a new query?
- When should the system clarify, refuse or escalate?
- How long should session context be retained?
- How are conversation state and context changes logged?

Typical outputs

- Conversation state rules.
- Follow-up handling rules.
- Context inheritance rules.
- Clarification and refusal rules.
- Session retention assumptions.
- Conversation logging requirements.
- Known context-risk scenarios.

Red flags

- Follow-ups silently inherit filters, periods or entities.
- Users can move outside scope through gradual follow-ups.
- The system answers from stale prior results.
- Context retention rules are unclear.
- Sensitive context is retained longer than needed.
- Conversation state is not visible in logs.
- Refusal rules reset or weaken across turns.

Practitioner note

Note 1: Multi-turn conversation is one of the main places where T2D risk increases. A follow-up may look harmless but change the metric, period, entity, permission scope or level of detail. The system should restate material inherited assumptions and revalidate context before answering.

Note 2: Multi-turn handling is also where costs can increase sharply. Each follow-up may trigger context review, metadata retrieval, model calls, validation, re-querying and answer generation. The design should decide when to reuse prior results, when to re-query, and when to stop or clarify rather than repeatedly expanding the conversation.

5.9 Design model and orchestration evaluation

Purpose: Define how the model choices and orchestration flow will be tested before and after prototype build.

Key design decisions

- Which tasks need evaluation: retrieval, query generation, validation, answer generation or follow-up handling?
- What test questions, expected answers and failure cases are required?
- Which dimensions will be scored: correctness, safety, latency, cost, exposure and usability?
- How will different models, prompts or routing patterns be compared?
- What quality threshold is required before progressing?
- How will evaluation results be logged, reviewed and approved?

Typical outputs

- Evaluation approach.
- Test-question set.
- Expected-answer set.
- Model comparison approach.
- Quality thresholds.
- Failure-case catalogue.

- Evaluation logging requirements.

Red flags

- Evaluation relies only on demo questions.
- Test cases exclude ambiguity, refusal and edge cases.
- Models are compared without fixed inputs or scoring criteria.
- Answer quality is judged only by fluency.
- Cost, latency and exposure are not scored.
- There is no threshold for rejecting a model or prompt.
- Evaluation results are not linked to change approval.

Practitioner note

Evaluation should test the orchestration, not only the model. A fluent answer is not enough: the system must retrieve the right metadata, generate or avoid queries appropriately, validate safely, preserve caveats, handle follow-ups and refuse when required. Model choice should be based on evidence across quality, safety, cost and latency, not preference or provider familiarity.

5.10 Design quality monitoring and improvement loop

Purpose: Define how answer quality, usage, failures, feedback and drift will be monitored after build, and how improvement actions will be governed.

Key design decisions

- Which quality signals should be monitored after launch?
- How will user feedback be captured and categorised?
- Which failures should trigger review, remediation or rollback?
- How will model, prompt, metadata and validation changes be tracked?
- What thresholds should trigger escalation or governance review?
- Who owns triage, prioritisation and improvement actions?

Typical outputs

- Quality monitoring approach.
- Feedback capture design.
- Usage and failure metrics.
- Drift and regression triggers.
- Improvement backlog process.
- Governance review thresholds.
- Quality reporting requirements.

Red flags

- Feedback is collected but not reviewed.
- Usage is measured without quality signals.
- Wrong answers cannot be traced to root cause.
- Model, prompt or metadata changes are not linked to quality movement.
- No one owns issue triage or improvement actions.
- Quality thresholds are unclear.

- Monitoring starts only after pilot or production.

Practitioner note

Quality monitoring is not only operational reporting. It is the mechanism that tells the team when the orchestration design needs to evolve. Feedback, failed validations, refusals, repeated clarifications, cost spikes, latency issues and regression failures should feed a governed improvement loop, not disappear into disconnected logs.

5.11 Define orchestration governance and change control

Purpose

Define how changes to the orchestration layer will be owned, approved, tested and released after prototype build.

Key design decisions

- Who owns model, prompt, tool, validation and metadata-contract changes?
- Which changes require approval before release?
- What evidence is required before changing prompts, models or tools?
- How are quality, cost, security and user-impact risks assessed?
- When should a change trigger regression testing or rollback?
- How are decisions, versions and approvals logged?

Typical outputs

- Orchestration governance model.
- Change-control rules.
- Approval ownership.
- Regression trigger rules.
- Rollback rules.
- Release evidence requirements.
- Governance logging requirements.

Red flags

- Prompt, model or tool changes can be made informally.
- Change ownership is unclear.
- Evaluation results are not required before release.
- Security and cost impacts are not reviewed.
- No rollback route exists.
- Metadata-contract changes are not coordinated with Phase 3 owners.
- Governance is only defined for production, not MVP or pilot.

Practitioner note

A T2D system will change often: prompts, models, metadata, validation rules, tools and thresholds will all evolve. Governance should make change safe, not slow every improvement. The goal is a clear route for approving, testing, releasing and rolling back orchestration changes without weakening trust or control.

6 Orchestration design decision pack

Phase 4 should produce a clear orchestration design pack that can guide bounded prototype build, evaluation, security review and later production planning. The pack should be detailed enough to avoid improvising critical runtime decisions during Phase 5, but not so detailed that it becomes a full production architecture document.

6.1 Orchestration design pack

The main output of Phase 4 is an orchestration design pack. It should include:

Output	Purpose
End-to-end orchestration flow	Shows how a user question becomes a governed answer.
Metadata contract	Defines which Phase 3 artefacts are needed at runtime.
Model, prompt and routing strategy	Defines which models are used for which tasks and why.
Tool registry and boundaries	Defines allowed tools, permissions, limits and failure behaviour.
Query generation and validation pattern	Defines how queries are created, checked, revised, blocked or escalated.
Answer and safe-failure rules	Defines how answers, caveats, refusals and escalations are handled.
Conversation handling rules	Defines how follow-ups reuse context, clarify or re-query.
Evaluation approach	Defines test sets, scoring dimensions, thresholds and comparison approach.
Quality monitoring approach	Defines feedback, usage, failure, drift and improvement signals.
Governance and change-control model	Defines ownership, approvals, regression triggers and rollback rules.
Security, cost and scale assumptions	Defines the constraints that shape model, tool and execution choices.
Open risks and decisions	Captures unresolved design risks, assumptions and owners.

For a POC, the pack may be lightweight. For an MVP, pilot or production path, it should be versioned, reviewed and strong enough to support security validation, regression testing and operational handover.

6.2 Output quality test

Before Phase 4 closes, the team should test whether the design is usable by asking:

- Can the prototype team build from this without inventing the orchestration pattern?
- Are model roles, tool boundaries and validation checks explicit?
- Can Phase 3 artefacts be retrieved, interpreted and applied at runtime?
- Are security, exposure, retention, cost and scale assumptions clear?
- Are clarification, refusal, escalation and safe-failure behaviours defined?
- Are follow-up questions controlled rather than treated as free-form memory?
- Can answer quality, model performance, cost and failures be evaluated?
- Can issues be traced from answer back to query, metadata, model and tool calls?
- Is there a clear route to approve, test and roll back orchestration changes?

If the answer is no to several of these questions, Phase 4 should not be treated as complete. The team should refine the design, narrow the scope or route unresolved issues back to the appropriate owner.

7 Exit criteria and handover

Phase 4 should close with an explicit decision on whether the orchestration design is ready for bounded prototype build. The exit decision should be based on design evidence, not on confidence that issues can be solved later.

7.1 Required exit outputs

The required exit output is the orchestration design decision pack described in Section 6, completed to the standard required for the delivery stage.

The exit decision should confirm whether the design can proceed, proceed with constraints, refine foundation, redesign or pause. It should also identify which risks, assumptions, design gaps and ownership issues remain open.

Note: Detailed UX design sits outside Phase 4, but Phase 4 must define the interaction behaviours that affect orchestration safety.

7.2 Handover to later phases

Handover area	What Phase 4 should provide

Prototype build	Clear flow, tool boundaries, model routing and build constraints for Phase 5.
Evaluation	Test sets, expected outputs, scoring dimensions and quality thresholds.
Security review	Exposure assumptions, access model, tool permissions, logs and retention rules.
Data / semantic owners	Metadata gaps, contract changes and remediation needs.
Product owner	Interaction assumptions, refusal behaviour, feedback capture and prototype limits.
Operating owner	Monitoring signals, issue triage route, governance cadence and ownership model.
Later production planning	Design risks, scale assumptions, cost controls and change-control requirements.

Some Phase 5 work may start before all Phase 4 details are complete, especially scaffolding, test harnesses or low-risk technical spikes. However, the prototype should not lock in model routes, tool permissions, query execution patterns, answer behaviour or governance rules before the relevant Phase 4 decisions are approved.

For Phase 5, the handover should include the intended orchestration flow, model/task assumptions, metadata contract, validation points, tool boundaries, logging requirements, cost and latency assumptions, governance rules and agreed prototype simplifications.

7.3 Exit decision wording

The exit decision should be stated clearly. Suggested wording:

Decision	Suggested wording
Proceed	The orchestration design is approved for bounded prototype build.
Proceed with constraints	Build may proceed only within agreed scope, tool, model or data limits.
Refine foundation	Phase 3 artefacts must be improved before reliable orchestration can proceed.
Redesign	The orchestration pattern needs material redesign before build.
Pause	Material risks remain unresolved and build should not proceed.

A proceed decision does not mean the design is production-ready. It means the orchestration is clear enough to build, test and learn from a bounded prototype.

7.4 Practitioner note

The handover from Phase 4 to Phase 5 should protect the prototype from becoming the architecture. A prototype can simplify the design, but it should not bypass the core controls: governed metadata, bounded tools, query validation, safe-failure behaviour, logging, evaluation and change control.

8 Key risks and failure modes

Phase 4 risks usually appear later as impressive prototypes that cannot be trusted, explained, secured, evaluated or operated. The main failure modes are:

Risk	Failure mode
Prompt-led governance	Business logic, joins, filters or caveats are left to the model instead of governed artefacts.
Weak metadata contract	Phase 3 artefacts exist but cannot be reliably retrieved, versioned or applied at runtime.
Over-broad tools	The assistant can access or execute more than the use case requires.
Shallow validation	Queries are checked for syntax but not for scope, permission, grain, joins, cost or exposure.
Unsafe answer generation	The final answer drops caveats, hides assumptions or overstates uncertain results.
Context drift	Follow-up questions inherit the wrong filters, periods, entities or permissions.
Poor evaluation	Model choice is based on demos rather than test sets, failure cases and thresholds.
No quality loop	Feedback, failures, cost and latency are logged but not reviewed or acted on.
Unclear governance	Prompt, model, tool or validation changes can happen without ownership, testing or rollback.
Prototype lock-in	Phase 5 hard-codes shortcuts that become difficult to replace later.

8.1 Practitioner note

The most dangerous Phase 4 failure is not choosing the wrong model. It is designing a flow where the model has too much discretion, the tools are too broad, the metadata is too weak, and the team cannot explain why an answer was produced. A good Phase 4 makes those risks visible before the prototype makes them look acceptable.